Node.js Event Loop

Output-Based Interview Practice | 11 Questions | Basic to Advanced

Student name: _____ Date: _____

Instructions: For each question, predict the console output in order and write a brief dry run. Identify when output order is not guaranteed. Assume separate runs and Node.js CommonJS (.js) unless a question explicitly asks for CommonJS and ES modules.

EASY

Q01. Regular functions and execution order

Predict the exact output. Explain how the function calls and call stack affect the order.

```
function first() {
  console.log("first");
}

function second() {
  console.log("second");
}

function third() {
  console.log("third");
}

first();
second();
third();
```

Your predicted output: _____

Reasoning / queue trace: _____

EASY

Q02. Synchronous code vs Promise vs timer

Predict the exact output and list the execution order of each callback.

```
console.log("A");

setTimeout(() => console.log("B"), 0);

Promise.resolve().then(() => console.log("C"));

console.log("D");
```

Your predicted output: _____

Reasoning / queue trace: _____

EASY–MEDIUM

Q03. Mixing nextTick, Promise, timer, and immediate

Predict the output. If any part of the output order can vary, identify the part and explain why.

```
console.log("start");

setTimeout(() => console.log("timeout"), 0);

setImmediate(() => console.log("immediate"));

Promise.resolve().then(() => console.log("promise"));

process.nextTick(() => console.log("tick"));

console.log("end");
```

Your predicted output: _____

Reasoning / queue trace: _____

MEDIUM

Q04. Nested Promise callbacks

Predict the exact output. Track the Promise microtask queue after each callback runs.

```
console.log("A");

Promise.resolve().then(() => {
  console.log("B");

  Promise.resolve().then(() => console.log("C"));
});

Promise.resolve().then(() => console.log("D"));

console.log("E");
```

Your predicted output: _____

Reasoning / queue trace: _____

MEDIUM

Q05. nextTick scheduled inside a Promise

Predict the exact output. Show the next-tick and Promise queues at each step.

```
console.log("Start");

Promise.resolve().then(() => {
  console.log("P1");
  process.nextTick(() => console.log("N2"));
});

process.nextTick(() => console.log("N1"));

Promise.resolve().then(() => console.log("P2"));

console.log("End");
```

Your predicted output: _____

Reasoning / queue trace: _____

MEDIUM–HARD

Q06. Top-level timer vs file I/O vs immediate

Predict the output order. State which ordering is guaranteed and which may depend on scheduling.

```
const fs = require("node:fs");

setTimeout(() => console.log("timeout"), 0);

fs.readFile(__filename, () => {
  console.log("file");
});

setImmediate(() => console.log("immediate"));
```

Your predicted output: _____

Reasoning / queue trace: _____

MEDIUM–HARD

Q07. Timer vs immediate inside an I/O callback

Predict the output. Explain how the callback context affects the ordering.

```
const fs = require("node:fs");

fs.readFile(__filename, () => {
  console.log("file");

  setTimeout(() => console.log("timeout"), 0);

  setImmediate(() => console.log("immediate"));
});
```

Your predicted output: _____

Reasoning / queue trace: _____

HARD

Q08. Full mixed event-loop example

Dry-run the code from top to bottom. Separate the synchronous output, queued microtasks, and event-loop callbacks. Identify any order that is not guaranteed.

```
const fs = require("node:fs");

console.log("start");

setTimeout(() => console.log("timeout"), 0);

fs.readFile(__filename, () => {
  console.log("file");
});

Promise.resolve().then(() => console.log("promise"));

process.nextTick(() => console.log("tick"));

setImmediate(() => console.log("immediate"));

console.log("end");
```

Your predicted output: _____

Reasoning / queue trace: _____

MEDIUM–HARD

Q09. async/await vs Promise microtasks

Predict the exact output. Mark which statements execute synchronously and which resume asynchronously.

```
async function demo() {
  console.log("A");

  await Promise.resolve();

  console.log("B");
}

console.log("C");

demo();

console.log("D");

Promise.resolve().then(() => console.log("E"));
```

Your predicted output: _____

Reasoning / queue trace: _____

HARD

Q10. CommonJS vs ES modules: nextTick and microtasks

Run the same code twice: once as a CommonJS file named index.cjs and once as an ES module named index.mjs. Predict the output for each run and explain any difference.

```
console.log("sync");

process.nextTick(() => console.log("nextTick"));

Promise.resolve().then(() => console.log("promise"));

queueMicrotask(() => console.log("microtask"));
```

Your predicted output: _____

Reasoning / queue trace: _____

HARD

Q11. Five-second timer vs immediate with queued callbacks

Predict the full output order. Explain why the timer delay matters and how the next-tick queue, Promise queue, Check phase, and timer interact.

```
console.log("Start");

setTimeout(() => console.log("Timeout 5 seconds"), 5000);

setImmediate(() => console.log("Immediate"));

Promise.resolve().then(() => console.log("Promise"));

process.nextTick(() => console.log("Tick"));

console.log("End");
```

Your predicted output: _____

Reasoning / queue trace: _____